

CS3601 操作系统 ChCore Lab 2

王凯灵 521030910356

2023.11.1

练习题 1

完成 `kernel/mm/buddy.c` 中的 `split_chunk`、`merge_chunk`、`buddy_get_pages`、和 `buddy_free_pages` 函数中的 LAB 2 TODO 1 部分，其中 `buddy_get_pages` 用于分配指定阶大小的连续物理页，`buddy_free_pages` 用于释放已分配的连续物理页。

要实现这些函数首先需要看懂函数 `get_buddy_chunk` 和 `init_buddy`。阅读代码，发现实现时使用 `page` 结构体进行管理。在初始化时，将一定范围的物理地址按单页大小初始化到 `page` 中，每页对应一个结构体。初始化时，使用了需要实现的 `buddy_free_pages`，所以首先进入这个函数。

`buddy_free_pages` 的目的是释放 `page` 并尽可能的向上合并。用于初始化时，其实会直接向上合并到最大。就结果而言，初始化结束后 free list 里应该只剩最大的块。事实上，笔者认为直接在 `init` 过程中完成大块的初始化更加自然和高效。然而，给出的不可改动部分指定了初始化阶段所有 `page` 状态为 `allocated`。这意味着我们并不能在释放第一个 `page` 时就递归的合并所有 `page`，而是在释放到 `buddy` 时再向前合并。每次最多只向上合并一层。这使得理解更加困难，也并不优雅。

由于需要用到合并，首先实现 `merge_chunk` 比较自然。`merge_chunk` 的实现逻辑为：使用 `get_buddy_chunk` 获取好哥们，如果哥们存在且有空，则合 ♂ 体及更新 free list 等，然后递归向上。这里需要注意递归中止条件是小于 `BUDDY_MAX_ORDER` 而不是等于，按头文件的定义。

然后就是 `buddy_free_pages`，解除 `allocated` tag，进行 `merge`，后更新 free list 即可。

`split_chunk` 拆分思路与合并类似，`order-1` 后将产生的 `buddy` 加入 free list。我的实现还检查了大部分异常情况。

最后 `buddy_get_pages`，寻找高于所需 `order` 的最小 free 块并 `split` 即可。一切要求实现的代码都贴在附录中。

由于不知道未来上层代码会不会异常处理，我使用 `BUG_ON` 宏处理了所有分配失败。

练习题 2

完成 `kernel/mm/slab.c` 中的 `choose_new_current_slab`、`alloc_in_slab_impl` 和 `free_in_slab` 函数中的 LAB 2 TODO 2 部分，其中 `alloc_in_slab_impl` 用于在 slab 分配器中分配指定阶大小的内存，而 `free_in_slab` 则用于释放上述已分配的内存。

首先，笔者愚笨，并未一眼看出 `choose_new_current_slab` 的作用。此函数在 `try_return_slab_to_buddy` 中调用，得知此函数用于将当前 slab 指向一个新 slab。于是只需要从 pool 的 partial list 中有则取之即可。

`alloc_in_slab_impl` 已经检查了当前 Slab 是否为空。TODO 部分尝试取出一个 slot，如果失败，则 new 一个，再失败就解锁并返回 NULL。否则正常取出并更新状态。

相比之下，`free_in_slab` 的操作就很简单，只需更新状态即可。

练习题 3

完成 `kernel/mm/kmalloc.c` 中的 `_kmalloc` 函数中的 LAB 2 TODO 3 部分, 在适当位置调用对应的函数, 实现 `kmalloc` 功能

只需要分别使用 `slab` 和 `buddy` 即可。其中由于本实现的 `buddy` 大小完全基于 `order`, 按理说需要更新实际大小到上层结构体中, 但并没有提供接口, 故忽略。

至此, `grade` 得到了 30 分。

练习题 4

完成 `kernel/arch/aarch64/mm/page_table.c` 中的 `query_in_pgtbl`、`map_range_in_pgtbl_common`、`unmap_range_in_pgtbl` 和 `mprotect_in_pgtbl` 函数中的 LAB 2 TODO 4 部分, 分别实现页表查询、映射、取消映射和修改页表权限的操作, 以 4KB 页为粒度。

首先, 清楚各个参数的定义。查看 `vmospace` 结构体得知 `pgtbl` 是页表基地址, 查看 `get_next_ptp`, 该函数修改传入的页表指针并返回指向类型。返回四种, 无映射、无内存、指向块、指向下一级。两个错误都返回负数, 直接判断小于 0 并将返回值返回即可。于是 `query_in_pgtbl` 的实现就是逐级调用 `get_next_ptp`, 报错则直接返回, 否则根据大块或正常映射修改 `pa` 指针。

实现时, 遇到了奇怪的地方, 就是在头文件中提供的宏不能支持 L0 页表项指向大块, 想来是用的太少不予支持。

`map_range_in_pgtbl_common` 需要范围映射。实现时我意识到其实 `get_next_ptp` 的当前地址和下一地址其实可用同一指针, 逻辑上不会出错, 但由于可读性差, 不采用循环写法。观察传入参数和被调用处, 推测函数功能就是从给定物理地址和虚拟地址的起点开始映射连续的长度。过程 L3 之前的过程中, `rss` 增加交给 `get_next_ptp` 处理即可。找到 L3 页表页时, 由于一般分配连续页, 所以直接顺着 L3 往下写, 直到写完当前的 L3 或达到要求。由于一个 L3 写满已经可以分配相当多内存, 不必对 L2 也做类似处理。这样总体上减少了页表页查询次数。

原先由于我思考分配物理页时应该要增加 `rss`, 一直不能通过测试。我把我的代码抄写了一份, 却能通过, 对比大仙只有 `rss` 不同。我确信错误的 `rss` 更新会导致一些问题, 而不是像描述中说的无需正确实现。既然如此, `rss` 可能代表的是虚拟内存使用情况, 而无需在 `map` 时增加。我没有仔细研究这一过程。

`unmap_range_in_pgtbl` 和 `mprotect_in_pgtbl`, 实现和上个函数思路完全一致。不做重复。至此, `grade` 已经给出了 70 分。

思考题 5

阅读 *Arm Architecture Reference Manual*, 思考要在操作系统中支持写时拷贝 (Copy-on-Write, CoW) 需要配置页表描述符的哪个/哪些字段, 并在发生页错误时如何处理。(在完成第三部分后, 你也可以阅读页错误处理的相关代码, 观察 `ChCore` 是如何支持 CoW 的)

我阅读了 *Arm Architecture Reference Manual* [1], 但我并没有发现里面有提到 CoW 的内容。的确, CoW 应该是由软件来支持的功能, 不归硬件负责。有种被忽悠的感觉。

抛开 Arm 架构, 想要实现写时拷贝, 直观上需要在写入一个地址时, 首先获知该内存是否允许写入, 还有该内存是否能被直接写入。

ARMv8 (此版本适用于 ARMv6 及以后) 提供了读写权限位 [1]。CoW 需要将 AP 设为 11 以保证严格只读。然后, 操作系统处理该异常, 确认是 CoW 操作还是非法写入, 可以通过在页表项中自定义一个 `flag` 实现。处理非法写入时, 如果是 CoW 操作, 就复制并重新处理虚拟空间。

Table D8-42 Data access permissions for a stage 1 translation supporting two Exception levels

AP[2:1]	Access from higher Exception level	Access from EL0
00	Read/write	None
01	Read/write	Read/write
10	Read-only	None
11	Read-only	Read-only

图 1: ARMv8 的定义 AP 位

看了第三部分以后，我确定 ChCore 的确这样操作。VMR 中维护了 VMR_COW 这一信息，用于识别非法写入错误需要 CoW 处理。接下来就是从页表项获取错误的页，创建一个新的虚拟段复制一份该页面，更新权限即可。然后还刷新了 TLB，因为页表更新了。

思考题 6

为了简单起见，在 ChCore 实验 Lab1 中没有为内核页表使用细粒度的映射，而是直接沿用了启动时的粗粒度页表，请思考这样做有什么问题。

粗粒度映射是没问题的，只是会造成浪费。毕竟我们专门写了 slab 机制给内核分配小块内存，内核往往需要使用已知大小的 4k 以内小块内存。使用大粒度会造成严重浪费。

挑战题 7

使用前面实现的 page_table.c 中的函数，在内核启动后的 main 函数中重新配置内核页表，进行细粒度的映射。

我们第一次分配页表是在进入 main 之前，完成 c 初始化时。进入 main 以后，我们很快就开始初始化 buddy 和 slab。重置页表应该在此操作之前。然而，经过分析，我发现 main 函数中并没有提供一些物理地址，偏置量，TTBR0 等重要宏（这些宏写在了 mmu.c 中，显然没有为所需实现做准备），且之前的实现完全不足以进行该操作。重置页表的实现是，首先 unmap 已经分配的页表，然后 map 新的页表，这需要对块分配的识别。

练习题 8

完成 kernel/arch/aarch64/irq/pgfault.c 中的 do_page_fault 函数中的 LAB 2 TODO 5 部分，将缺页异常转发给 handle_trans_fault 函数。

好像就只是转发，

```
1 ret = handle_trans_fault(current_thread->vmSPACE, fault_addr);
```

练习题 9

完成 kernel/mm/vmSPACE.c 中的 find_vmr_for_va 函数中的 LAB 2 TODO 6 部分，找到一个虚拟地址在其虚拟地址空间中的 VMR。

该函数可以顾名思义，从虚拟空间获取虚拟地址所在的虚拟段，所以无需看懂整个文件，谢天谢地。课上所述的虚拟空间使用链表组织，这里使用红黑树组织，直接调用红黑树查找即可。

```
1 struct rb_node *node = rb_search(&vmSPACE->vmr_tree, (const void *)addr,
    cmp_vmr_and_va);
2 if (node) {
```

```

3     return rb_entry(node, struct vmregion, tree_node);
4 }
5 return NULL;

```

练习题 10

完成 `kernel/mm/pgfault_handler.c` 中的 `handle_trans_fault` 函数中的 LAB 2 TODO 7 部分（函数内共有 3 处填空，不要遗漏），实现 PMO_SHM 和 PMO_ANONYM 的按需物理页分配。你可以阅读代码注释，调用你之前见到过的相关函数来实现功能。

第一处，获得一个页置 0。

```

1 pa = virt_to_phys(get_pages(0));
2 memset(phys_to_virt(pa), 0, PAGE_SIZE);

```

后两处都是分配，调用即可。之前实现的函数没有在头文件给出，但是有一个转接函数。

挑战题 11

由于没有磁盘，因此我们采用一部分内存模拟磁盘。内存页是可以换入换出的，请设计一套换页策略（如 LRU 等），并在 `kernel/mm/pgfault_handler.c` 中的适当位置简单实现你的换页方法。

这个实现完全是按照已有策略的工程师工作，且还需要另外准备测试函数，故我决定不实现。

参考文献

[1] ARM. Arm® Architecture Reference Manual Armv8, for Armv8-A architecture profile. [Online]. Available: <https://developer.arm.com/documentation/ddi0487/latest/>. [Accessed: Nov. 1, 2023].

附录

```

1 static struct page *split_chunk(struct phys_mem_pool *pool, int order,
2 struct page *chunk)
3 {
4     /* LAB 2 TODO 1 BEGIN */
5     /*
6      * Hint: Recursively put the buddy of current chunk into
7      * a suitable free list.
8      */
9     /* BLANK BEGIN */
10    /* A good coding engineer should write handful comments, I was told. */
11    struct page *buddy_chunk;
12    struct free_list *free_list;
13
14    // BUG_ON(chunk->order < order || chunk->order >= BUDDY_MAX_ORDER); // Could this
        happen? In case, anyway.
15
16    /* End of recursive. */

```

```

17     if (chunk->order == order) {
18         return chunk;
19     }
20
21     /* Split the chunk into two smaller chunks. */
22     chunk->order--;
23     buddy_chunk = get_buddy_chunk(pool, chunk); // The buddy chunk is the rest of the
24     input chunk.
25     BUG_ON(buddy_chunk == NULL); // In fact this should never happen if the input
26     chunk is legal.
27     buddy_chunk->order = chunk->order;
28
29     /* Add the buddy chunk to the free list. */
30     free_list = &pool->free_lists[chunk->order];
31     list_add(&buddy_chunk->node, &free_list->free_list);
32     free_list->nr_free++;
33
34     /* Recursively split the chunk. */
35     return split_chunk(pool, order, chunk);
36     /* BLANK END */
37     /* LAB 2 TODO 1 END */
38 }

```

```

1 static struct page *merge_chunk(struct phys_mem_pool *pool, struct page *chunk)
2 {
3     /* LAB 2 TODO 1 BEGIN */
4     /*
5      * Hint: Recursively merge current chunk with its buddy
6      * if possible.
7      */
8     /* BLANK BEGIN */
9     struct page *buddy_chunk;
10    struct list_head *free_list;
11
12    BUG_ON(chunk->order >= BUDDY_MAX_ORDER); // In case.
13
14    /* End of recursive. */
15    if (chunk->order == BUDDY_MAX_ORDER - 1){
16        return chunk;
17    }
18
19    /* Get the buddy chunk of the current chunk. */
20    buddy_chunk = get_buddy_chunk(pool, chunk);
21    /* Must be a legal buddy, but the grading seems to require us to return. */
22    // BUG_ON(buddy_chunk == NULL);
23
24    /* Check if the buddy chunk is free. */
25    if (buddy_chunk != NULL && !buddy_chunk->allocated && buddy_chunk->order ==
26        chunk->order) {
27        /* Update free list. */
28        free_list = &(pool->free_lists[chunk->order].free_list);
29        list_del(&buddy_chunk->node);
30        pool->free_lists[chunk->order].nr_free--;

```

```

30
31     /* Merge the buddy chunk with the current chunk. */
32     chunk = chunk > buddy_chunk ? buddy_chunk : chunk;
33     chunk->order++;
34
35     /* Recursively call merge_chunk. */
36     return merge_chunk(pool, chunk);
37 }
38 else {
39     return chunk;
40 }
41 /* BLANK END */
42 /* LAB 2 TODO 1 END */
43 }

```

```

1 struct page *buddy_get_pages(struct phys_mem_pool *pool, int order)
2 {
3     int cur_order;
4     struct list_head *free_list;
5     struct page *page = NULL;
6
7     if (unlikely(order >= BUDDY_MAX_ORDER)) {
8         kwarn("ChCore does not support allocating such too large "
9             "contiguous physical memory\n");
10        return NULL;
11    }
12
13    lock(&pool->buddy_lock);
14
15    /* LAB 2 TODO 1 BEGIN */
16    /*
17     * Hint: Find a chunk that satisfies the order requirement
18     * in the free lists, then split it if necessary.
19     */
20    /* BLANK BEGIN */
21    /* Seek from order. */
22    for (cur_order = order; cur_order < BUDDY_MAX_ORDER; ++cur_order) {
23        free_list = &(pool->free_lists[cur_order].free_list);
24        if (pool->free_lists[cur_order].nr_free) {
25            page = list_entry(free_list->next, struct page, node);
26            list_del(&page->node);
27            pool->free_lists[cur_order].nr_free--;
28            page = split_chunk(pool, order, page);
29            page->allocated = 1;
30            break;
31        }
32    }
33
34    // BUG_ON(page == NULL); // Failed to allocate memory.
35
36
37    /* BLANK END */
38    /* LAB 2 TODO 1 END */

```

```

39 out:
40 unlock(&pool->buddy_lock);
41 return page;
42 }

```

```

1 void buddy_free_pages(struct phys_mem_pool *pool, struct page *page)
2 {
3     int order;
4     struct list_head *free_list;
5
6     lock(&pool->buddy_lock);
7
8     /* LAB 2 TODO 1 BEGIN */
9     /*
10      * Hint: Merge the chunk with its buddy and put it into
11      * a suitable free list.
12      */
13     /* BLANK BEGIN */
14     /* Deal with freed page first. */
15     page->allocated = 0;
16     page = merge_chunk(pool, page);
17     order = page->order;
18     free_list = &(pool->free_lists[order].free_list);
19     list_add(&page->node, free_list);
20     pool->free_lists[order].nr_free++;
21     /* BLANK END */
22     /* LAB 2 TODO 1 END */
23
24     unlock(&pool->buddy_lock);
25 }

```

```

1 static void choose_new_current_slab(struct slab_pointer *pool, int order)
2 {
3     /* LAB 2 TODO 2 BEGIN */
4     /* Hint: Choose a partial slab to be a new current slab. */
5     /* BLANK BEGIN */
6     struct slab_header *slab;
7
8     if (list_empty(&pool->partial_slab_list)) {
9         slab = init_slab_cache(order, SIZE_OF_ONE_SLAB);
10        if (slab == NULL) {
11            pool->current_slab = NULL;
12            return;
13        }
14    } else {
15        slab = list_entry(pool->partial_slab_list.next, struct slab_header, node);
16        list_del(&slab->node);
17    }
18    pool->current_slab = slab;
19    /* BLANK END */
20    /* LAB 2 TODO 2 END */
21 }

```

```

1  static void *alloc_in_slab_impl(int order)
2  {
3      struct slab_header *current_slab;
4      struct slab_slot_list *free_list;
5      void *next_slot;
6
7      lock(&slabs_locks[order]);
8
9      current_slab = slab_pool[order].current_slab;
10     /* When serving the first allocation request. */
11     if (unlikely(current_slab == NULL)) {
12         current_slab = init_slab_cache(order, SIZE_OF_ONE_SLAB);
13         if (current_slab == NULL) {
14             unlock(&slabs_locks[order]);
15             return NULL;
16         }
17         slab_pool[order].current_slab = current_slab;
18     }
19
20     /* LAB 2 TODO 2 BEGIN */
21     /*
22      * Hint: Find a free slot from the free list of current slab.
23      * If current slab is full, choose a new slab as the current one.
24      */
25     /* BLANK BEGIN */
26     free_list = current_slab->free_list_head;
27     /* Try new and check. */
28     if (free_list == NULL) {
29         choose_new_current_slab(&slab_pool[order], order);
30         current_slab = slab_pool[order].current_slab;
31         if (current_slab == NULL) {
32             unlock(&slabs_locks[order]); // Don't forget this.
33             return NULL;
34         }
35         free_list = current_slab->free_list_head;
36     }
37
38     next_slot = free_list->next_free;
39     current_slab->free_list_head = next_slot;
40     current_slab->current_free_cnt--;
41     /* BLANK END */
42     /* LAB 2 TODO 2 END */
43
44     unlock(&slabs_locks[order]);
45
46     return (void *)free_list;
47 }

```

```

1  void free_in_slab(void *addr)
2  {
3      struct page *page;
4      struct slab_header *slab;
5      struct slab_slot_list *slot;

```



```

6     int order;
7
8     slot = (struct slab_slot_list *)addr;
9     page = virt_to_page(addr);
10    if (!page) {
11        kdebug("invalid page in %s", __func__);
12        return;
13    }
14
15
16    slab = page->slab;
17    order = slab->order;
18    lock(&slabs_locks[order]);
19
20    try_insert_full_slab_to_partial(slab);
21
22    #if ENABLE_DETECTING_DOUBLE_FREE_IN_SLAB == ON
23    /*
24     * SLAB double free detection: check whether the slot to free is
25     * already in the free list.
26     */
27    if (check_slot_is_free(slab, slot) == 1) {
28        kinfo("SLAB: double free detected. Address is %p\n",
29            (unsigned long)slot);
30        BUG_ON(1);
31    }
32    #endif
33
34    /* LAB 2 TODO 2 BEGIN */
35    /*
36     * Hint: Free an allocated slot and put it back to the free list.
37     */
38    /* BLANK BEGIN */
39    slot->next_free = slab->free_list_head;
40    slab->free_list_head = (void *)slot;
41    slab->current_free_cnt++;
42    /* BLANK END */
43    /* LAB 2 TODO 2 END */
44
45    try_return_slab_to_buddy(slab, order);
46
47    unlock(&slabs_locks[order]);
48 }

```

```

1 void *_kmalloc(size_t size, bool is_record, size_t *real_size)
2 {
3     void *addr;
4     int order;
5
6     if (unlikely(size == 0))
7         return ZERO_SIZE_PTR;
8
9     if (size <= SLAB_MAX_SIZE) {

```

```

10     /* LAB 2 TODO 3 BEGIN */
11     /* Step 1: Allocate in slab for small requests. */
12     /* BLANK BEGIN */
13     addr = alloc_in_slab(size, real_size);
14     /* BLANK END */
15     #if ENABLE_MEMORY_USAGE_COLLECTING == ON
16     if(is_record && collecting_switch) {
17         record_mem_usage(*real_size, addr);
18     }
19     #endif
20 } else {
21     /* Step 2: Allocate in buddy for large requests. */
22     /* BLANK BEGIN */
23     order = size_to_page_order(size);
24     addr = get_pages(order);
25     /* BLANK END */
26     /* LAB 2 TODO 3 END */
27 }
28
29 BUG_ON(!addr);
30 return addr;
31 }

```

```

1 int query_in_pgtbl(void *pgtbl, vaddr_t va, paddr_t *pa, pte_t **entry)
2 {
3     /* LAB 2 TODO 4 BEGIN */
4     /*
5      * Hint: Walk through each level of page table using `get_next_ptp`,
6      * return the pa and pte until a L0/L1 block or page, return
7      * `-ENOMAPPING` if the va is not mapped.
8      */
9     /* BLANK BEGIN */
10    ptp_t *l0_ptp, *l1_ptp, *l2_ptp, *l3_ptp, *vp;
11    pte_t *pte;
12    int ret;
13
14    /* L0 page table */
15    l0_ptp = (ptp_t *)pgtbl;
16    ret = get_next_ptp(l0_ptp, L0, va, &l1_ptp, &pte, false, NULL);
17    if (ret < 0) {
18        return ret;
19    }
20
21    /* L1 page table */
22    ret = get_next_ptp(l1_ptp, L1, va, &l2_ptp, &pte, false, NULL);
23    if (ret < 0) {
24        return ret;
25    }
26    else if (ret == BLOCK_PTP) {
27        if (entry != NULL) {
28            *entry = pte;
29        }
30        *pa = virt_to_phys(l2_ptp) + GET_VA_OFFSET_L1(va);

```

```

31     return 0;
32 }
33
34
35 /* L2 page table */
36 ret = get_next_ptp(l2_ptp, L2, va, &l3_ptp, &pte, false, NULL);
37 if (ret < 0) {
38     return ret;
39 }
40 else if (ret == BLOCK_PTP) {
41     if (entry != NULL) {
42         *entry = pte;
43     }
44     *pa = virt_to_phys(l3_ptp) + GET_VA_OFFSET_L2(va);
45     return 0;
46 }
47
48 /* L3 page table */
49 ret = get_next_ptp(l3_ptp, L3, va, &vp, &pte, false, NULL);
50 if (ret < 0) {
51     return ret;
52 }
53 if (entry != NULL) {
54     *entry = pte;
55 }
56 *pa = virt_to_phys(vp) + GET_VA_OFFSET_L3(va);
57 /* LAB 2 TODO 4 END */
58 return 0;
59 }

```

```

1  static int map_range_in_pgtbl_common(void *pgtbl, vaddr_t va, paddr_t pa, size_t len,
2  vmr_prop_t flags, int kind, long *rss)
3  {
4      /* LAB 2 TODO 4 BEGIN */
5      /*
6       * Hint: Walk through each level of page table using `get_next_ptp`,
7       * create new page table page if necessary, fill in the final level
8       * pte with the help of `set_pte_flags`. Iterate until all pages are
9       * mapped.
10      * Since we are adding new mappings, there is no need to flush TLBs.
11      * Return 0 on success.
12      */
13      /* BLANK BEGIN */
14      ptp_t *l0_ptp, *l1_ptp, *l2_ptp, *l3_ptp;
15      pte_t *pte;
16      int ret;
17
18      l0_ptp = (ptp_t *)pgtbl;
19      size_t allocated_size = 0;
20      vaddr_t cur_va = va;
21      paddr_t cur_pa = pa;
22
23      while (allocated_size < len) {

```

```

24
25     /* Find the L3. */
26     ret = get_next_ptp(l0_ptp, L0, cur_va, &l1_ptp, &pte, true, rss);
27     if (ret < 0) {
28         return ret;
29     }
30
31     ret = get_next_ptp(l1_ptp, L1, cur_va, &l2_ptp, &pte, true, rss);
32     if (ret < 0) {
33         return ret;
34     }
35
36     ret = get_next_ptp(l2_ptp, L2, cur_va, &l3_ptp, &pte, true, rss);
37     if (ret < 0) {
38         return ret;
39     }
40
41     /* Mapping. */
42     for (int ent_i = GET_L3_INDEX(cur_va); ent_i < PTP_ENTRIES; ++ent_i) {
43         pte_t t_pte;
44
45         t_pte.pte = 0;
46         t_pte.l3_page.is_valid = 1;
47         t_pte.l3_page.is_page = 1;
48         t_pte.l3_page.pfn = cur_pa >> PAGE_SHIFT;
49         set_pte_flags(&t_pte, flags, USER_PTE);
50         l3_ptp->ent[ent_i].pte = t_pte.pte;
51
52         /* A new page allocated in L3. */
53         allocated_size += PAGE_SIZE;
54         // *rss += PAGE_SIZE; // ??????wtf
55         cur_va += PAGE_SIZE;
56         cur_pa += PAGE_SIZE;
57         if (allocated_size >= len) {
58             break;
59         }
60     }
61 }
62 /* BLANK END */
63 /* LAB 2 TODO 4 END */
64 return 0;
65 }

```

```

1  int unmap_range_in_pgtbl(void *pgtbl, vaddr_t va, size_t len, long *rss)
2  {
3      /* LAB 2 TODO 4 BEGIN */
4      /*
5       * Hint: Walk through each level of page table using `get_next_ptp`,
6       * mark the final level pte as invalid. Iterate until all pages are
7       * unmapped.
8       * You don't need to flush tlb here since tlb is now flushed after
9       * this function is called.
10      * Return 0 on success.

```

```

11  */
12  /* BLANK BEGIN */
13  ptp_t *l0_ptp, *l1_ptp, *l2_ptp, *l3_ptp;
14  pte_t *pte;
15  int ret;
16
17  l0_ptp = (ptp_t *)pgtbl;
18  size_t unmapped_size = 0;
19  vaddr_t cur_va = va;
20
21  while (unmapped_size < len) {
22
23      /* Find the L3. */
24      ret = get_next_ptp(l0_ptp, L0, cur_va, &l1_ptp, &pte, false, rss);
25      if (ret < 0) {
26          return ret;
27      }
28
29      ret = get_next_ptp(l1_ptp, L1, cur_va, &l2_ptp, &pte, false, rss);
30      if (ret < 0) {
31          return ret;
32      }
33
34      ret = get_next_ptp(l2_ptp, L2, cur_va, &l3_ptp, &pte, false, rss);
35      if (ret < 0) {
36          return ret;
37      }
38
39      /* Freeing. */
40      for (int ent_i = GET_L3_INDEX(cur_va); ent_i < PTP_ENTRIES; ++ent_i) {
41          l3_ptp->ent[ent_i].pte = PTE_DESCRIPTOR_INVALID;
42
43          cur_va += PAGE_SIZE;
44          unmapped_size += PAGE_SIZE;
45          if (unmapped_size >= len) {
46              break;
47          }
48      }
49  }
50  /* BLANK END */
51  /* LAB 2 TODO 4 END */
52
53  dsb(ishst);
54  isb();
55
56  return 0;
57 }

```